CS 202
Advanced Operating Systems

Winter 26

Lecture 6: Scheduling

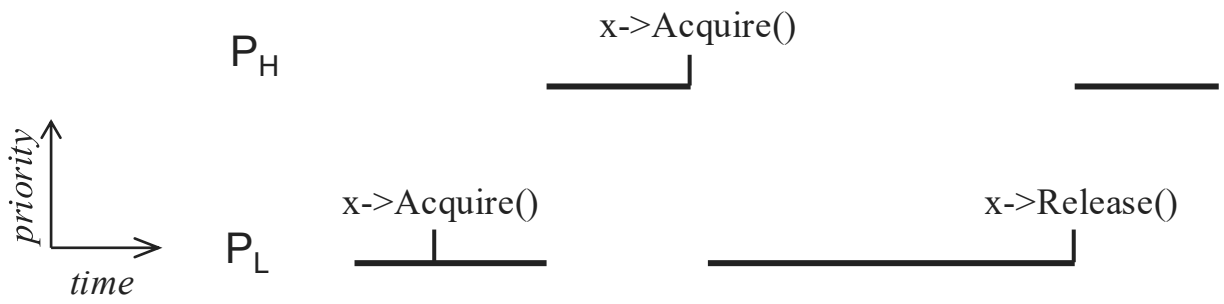
Instructor: Chengyu Song

Question

- Near full utilization is one of the most important goal of scheduling, “fake” idling is not good, so when/how can fake idling happen?

More on Priority Scheduling

- For real-time (predictable) systems, priority is often used to isolate a process from those with lower priority. *Priority inversion* is a risk unless all resources are jointly scheduled.

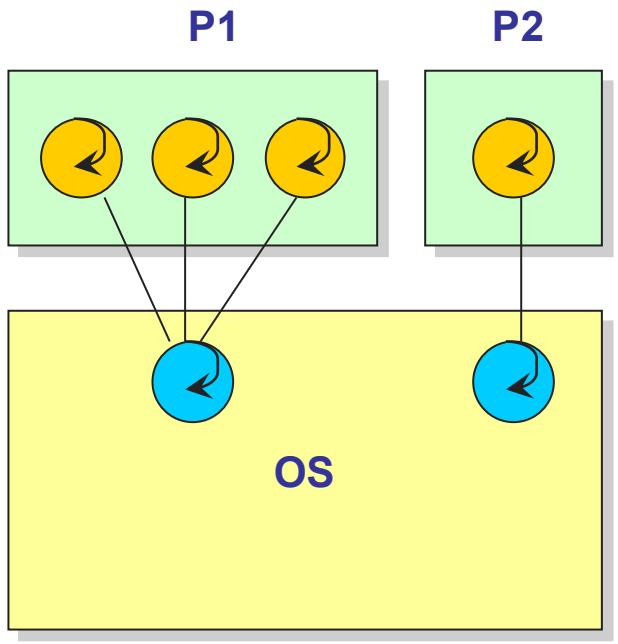


Priority Inheritance

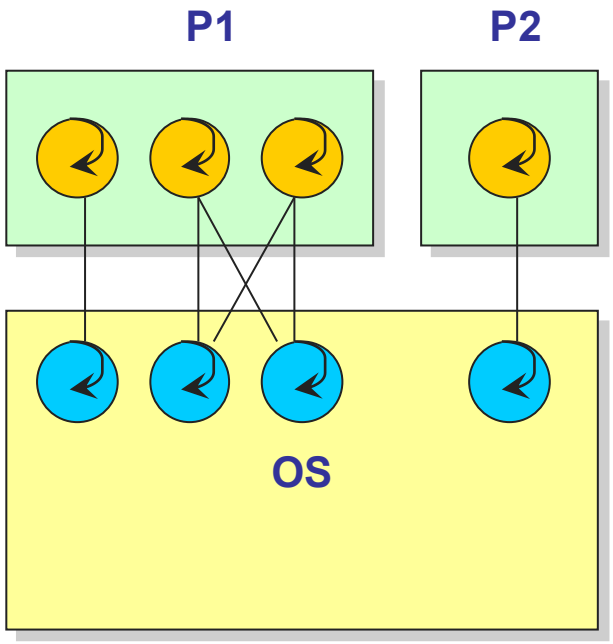
- If lower priority process is being waited on by a higher priority process it inherits its priority
  - How does this help?
  - Does it prevent the previous problem?

- Priority inversion is a big problem for real-time systems
  - Mars pathfinder bug (link)

User and Kernel Threads



Multiplexing user-level threads on a single kernel thread for each process



Multiplexing user-level threads on multiple kernel threads for each process

KLT vs. ULT

- Kernel-level threads
  - Integrated with OS (informed scheduling)
  - Slow to create, manipulate, synchronize
- User-level threads
  - Fast to create, manipulate, synchronize
  - Not integrated with OS (uninformed scheduling)
- Understanding the differences between kernel and user-level threads is important
  - For programming (correctness, performance)
  - For test-taking ☺

Scheduler Activations

- Scheduler Activations: Effective Kernel Support for the User-Level Management of Parallelism, Anderson, Bershad, Lazowska, Levy — SOSP '91, also TOCS ‘92
- Topic: how to get the best of both worlds between user-level threads (fast + flexible) and kernel threads (correct + integrated)
  - Pure user-level threads (M:1): context switch = procedure call (~10-50 cycles), scheduling policy/algorithm: flexible, program specific
  - **Flaws** in multiprogrammed multiprocessors
    - » The kernel treats the entire process as a black box, when one user thread does a blocking syscall, page fault, or I/O, the whole process blocks.
    - » No other user thread can run on that CPU, and you can't properly utilize multiple cores under load or multiprogramming.
  - Kernel-level threads (1:1): Kernel knows everything → perfect integration with I/O, signals, preemption, CPU accounting, multiprogramming fairness.
  - **Flaws**: Thread operations (create, switch, yield, sync) are 10-100× slower because they cross the kernel boundary every time. App-specific policies are painful.

Scheduler Activations

- Key insights:
  - The kernel doesn't need to know about every thread — it only needs to know about processor allocation and blocking events.
  - The user level doesn't need the kernel to schedule its threads — it just needs to be told when something important happens.
- Core idea: let the kernel scheduler and the user-level scheduler coordinate with each other
- Core abstraction: **scheduler activation**
  - A scheduler activation is not a full kernel thread. It is:
    - » A lightweight execution context (like a virtual CPU) allocated to the address space.
    - » The place where the kernel saves/restores the state of a user thread when it blocks or is preempted.
    - » User-level scheduler can then decide which thread it wants to run on that vCPU.

Scheduler Activations: coordinations

- **Downcalls**: user-level scheduler tells the kernel what it needs
  - "I need more processors" (when #runnable user threads > #activations)
  - "I have idle processors" (when #runnable < #activations)
  - "Recycle this activation" (cleanup)
- **Upcalls**: kernel calls user-level scheduler to notify it of events (**the new thing**)
  - "Add this processor" (new activation)
  - "Processor has been preempted" (lost activation + saved context of what was running)
  - "Scheduler activation has blocked" (e.g., syscall blocked, page fault) → kernel immediately creates a new activation and upcalls on it so the user scheduler can run something else
  - "Scheduler activation has unblocked" → delivers the now-runnable thread's saved context (often by preempting one of your current activations, so you get two contexts in the upcall)

Scheduler Activations: an example

- **Blocking** (e.g., read() blocks)
  - Current activation blocks → kernel creates new activation → upcall "activation blocked" on the new one → user scheduler picks another ready user thread and runs it immediately on the same physical CPU. No wasted cycles.
- **Unblocking**
  - The blocked thread becomes runnable → kernel does an upcall (possibly preempting another of your activations) → you now have two contexts: (a) the one that was running, and (b) the newly unblocked thread. You decide: keep running current? Switch? etc.
- **Preemption** (kernel takes a CPU away for another process)
  - Kernel stops the activation and upcalls "preempted" with the saved context → user scheduler moves that thread back to ready queue and continues with remaining activations.

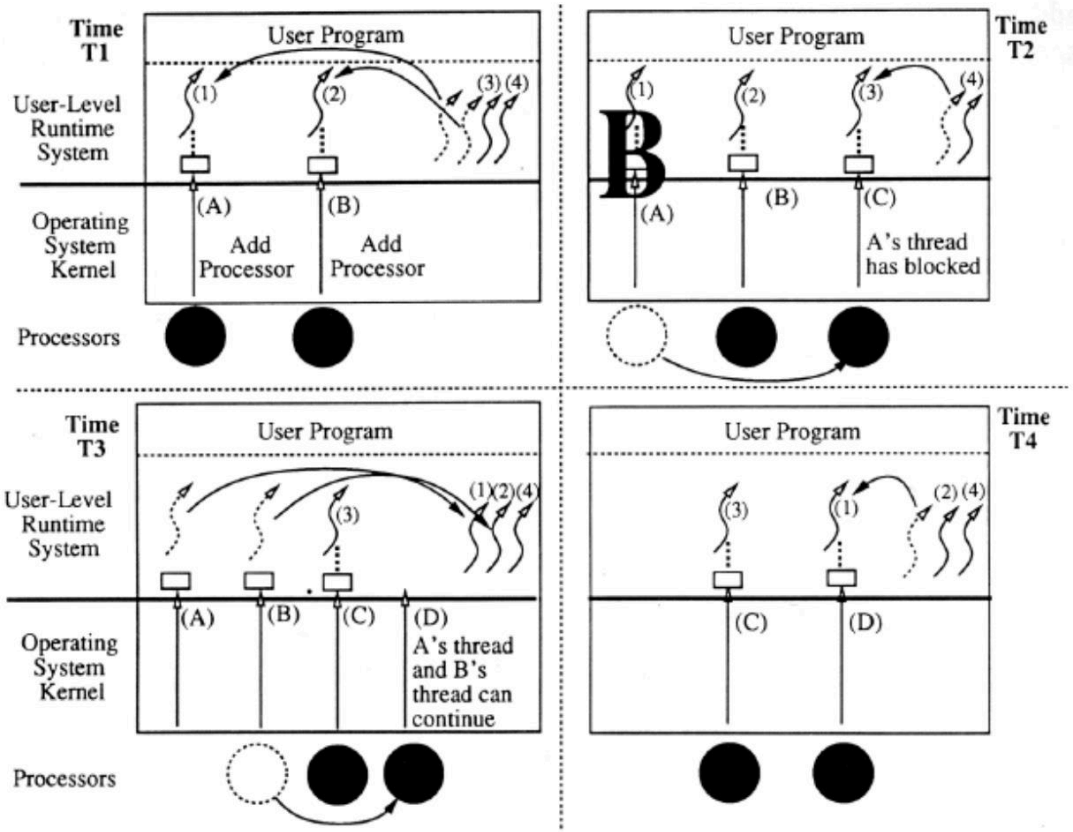


Fig. 1. Example: I/O request/completion.

Scheduler Activations: preemptions

- A tricky situation: what if you get an upcall while a user thread is in a critical section, e.g., holds a user-level spinlock in the runtime itself?
- Solution (very clever): **continuation/recovery**
  - Upcall handler checks (via a hash table of critical-section address ranges) whether the affected thread was inside a critical section.
  - If yes, temporarily continue that thread (on the upcall activation) until it exits the section, then resume normal scheduling.
  - No global preemption disabling, no priority inversion at user level.

Scheduler Activations: performance

- Scheduler activations added to Topaz kernel thread management.
  - Performs upcalls instead of own scheduling.
  - Explicit processor allocation to address spaces.
- Modifications to FastThreads user-level thread package
  - Processing of upcalls.
  - Resume interrupted critical sections.
  - Pass processor allocation information to Topaz.

Scheduler Activation: performance

- Thread performance without kernel involvement similar to FastThreads before changes.
- Upcall performance significantly worse than Topaz threads.
  - Untuned implementation.
  - Topaz in assembler, this system in Modula-2+.
- Application performance
  - Negligible I/O: As quick as original FastThreads.
  - With I/O: Performs better than either FastThreads or Topaz threads.

Scheduler Activations: performance

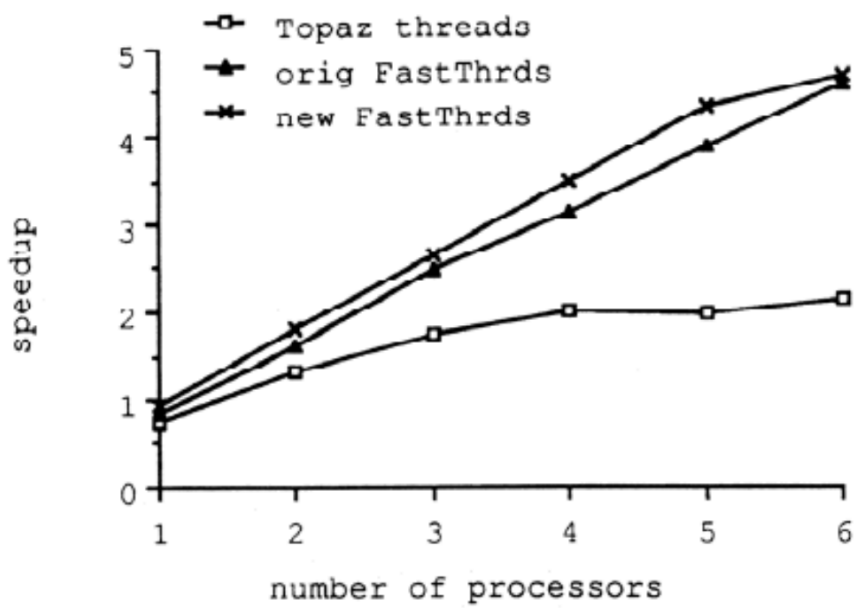


Fig. 2. Speedup of N-Body application versus number of processors, 100% of memory available.

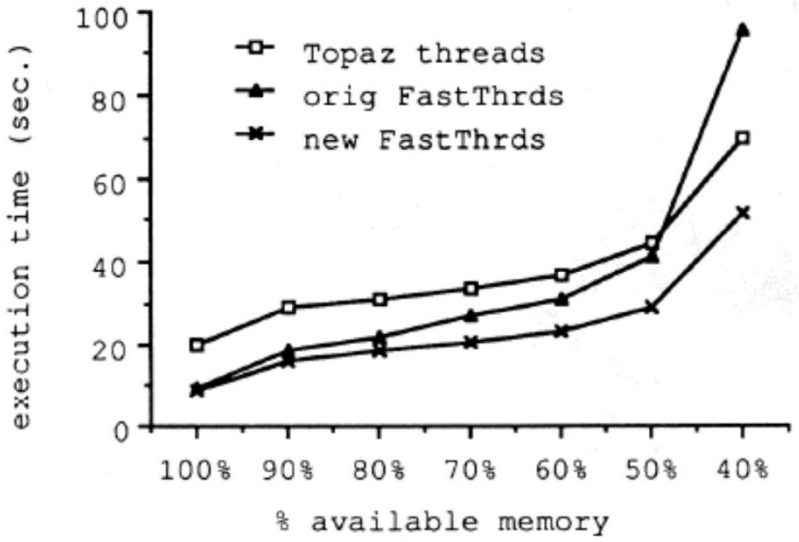


Fig. 3. Execution time of N-Body application versus amount of available memory, 6 processors.

Scheduler Activations: adoptions

- Not really adopted in real monolithic kernels due to its complexity
- But the ideas are adopted in other runtimes:
  - Go's goroutine scheduler (M:N with its own "processors").
  - Many language virtual threads (Project Loom in Java, etc.).
  - ghOSt (Google, SOSP 2021) — the modern spiritual successor: kernel safely delegates entire scheduling policy (even across processes) to user-space agents running in enclaves. Same philosophy: "give control to user level, but safely."